

Secure Multi-Tenant Kubernetes Platform on OpenNebula IaaS

Alessandro Brognara (261216), Lorenzo Bergami (266671)

August 10, 2026

Fog and Cloud Computing — Project Proposal
Computer Science Master Degree — University of Trento

1 Problem Statement and Motivation

Modern software companies rarely dedicate a separate physical or virtual infrastructure to each internal development team. Consolidating several teams onto a single shared cluster reduces operational cost and simplifies management, but it introduces a security problem that does not exist when workloads run in isolation: a shared platform is only as trustworthy as the boundaries drawn inside it.

We consider a realistic scenario: a small software house runs the web applications of two internal development teams, Team Alpha and Team Beta, on a single shared platform. Each team must be able to deploy and manage its own applications autonomously, without being able to observe, disturb, or reach the workloads of the other team. Neither team should be able to weaken the platform itself, for example by deploying privileged containers or by consuming resources without limits.

The project builds this platform end to end, combining an IaaS layer (OpenNebula) that provisions and network-isolates the virtual machines, with a PaaS layer (Kubernetes, in the lightweight k3s distribution) that orchestrates the containerised workloads and enforces tenant separation. Security is not an add-on: the isolation boundaries between the two tenants are the core technical contribution of the work.

2 Objectives

The project pursues four concrete objectives:

- **Provision infrastructure as code.** Use OpenNebula to create the virtual machines, the virtual network and the network-level access rules that constitute the cluster substrate.
- **Build a working multi-node Kubernetes cluster.** Bootstrap k3s on the provisioned VMs (one control-plane node, two worker nodes), rather than relying on a single-node development environment such as Minikube.
- **Implement defence in depth across two architectural layers.** Combine network-level controls at the IaaS layer with authorisation, network, runtime and admission controls at the Kubernetes layer.
- **Demonstrate the boundaries empirically.** Show, in a live demo, that each control actually blocks the attack it is meant to block, not merely that the platform works when used correctly.

3 Proposed Architecture

The architecture is organised in two stacked layers, each contributing its own isolation mechanisms.

3.1 IaaS layer — OpenNebula

OpenNebula runs on the lab virtual machine provided by the course (Ubuntu 24.04, 16 GB RAM, 4 vCPU) through the MiniONE deployment, using KVM as the underlying hypervisor. From this platform we provision three virtual machines connected to a dedicated OpenNebula virtual network:

- **k8s-control-plane**, hosts the k3s server (API server, scheduler, controller manager, datastore).
- **k8s-worker-1** and **k8s-worker-2**, host the k3s agents that execute the tenant workloads.

The choice of k3s over a full kubeadm deployment is driven by the resource budget: the entire cluster is nested inside a single 16 GB / 4 vCPU host that also runs OpenNebula itself. k3s provides a fully conformant Kubernetes API with a substantially smaller memory and CPU footprint, leaving room for a realistic three-node topology instead of a single overloaded node.

3.2 PaaS layer — Kubernetes tenancy model

Each development team is modelled as a Kubernetes namespace: **team-alpha** and **team-beta**. This is a soft multi-tenancy model, the two tenants share the same cluster and the same worker node kernels, and are separated logically rather than physically. This is a deliberate design decision and it matches the scenario: the tenants are two teams inside the same company, which trust each other partially, so the operational and resource cost of physical separation is not justified.

Within its own namespace, each team can create, inspect and delete its own Deployments, Services, ConfigMaps and Pods. Outside it, the team has no visibility and no privileges. A dedicated ServiceAccount and kubeconfig per tenant will be used during the demo to impersonate a team member.

Layer	Component	Role in the architecture
IaaS	OpenNebula + KVM	VM provisioning, virtual network, security groups
IaaS	Virtual network	Private L2/L3 connectivity between cluster nodes
PaaS	k3s (3 nodes)	Container orchestration and cluster API
PaaS	CNI plugin	Pod-to-pod networking and NetworkPolicy enforcement
PaaS	Namespaces	Tenant boundary for Team Alpha and Team Beta
PaaS	OPA / Gatekeeper	Custom admission-time policy validation (planned extension)

4 Security Design

Rather than selecting security mechanisms first and justifying them afterwards, we start from the threats that a shared platform introduces, and derive the controls from them.

4.1 Threat model

We assume the two tenants are non-malicious by default but potentially careless, and that any single workload may become compromised. Under this assumption the relevant threats are:

- **Cross-tenant data access.** A member of Team Alpha reads, modifies or deletes resources belonging to Team Beta through the Kubernetes API.
- **Cross-tenant network access.** A pod of one tenant reaches the database or the application of the other tenant directly over the pod network.
- **Privilege escalation from a workload.** A compromised container runs as root, requests privileged mode or host networking, and attempts to affect the node it runs on.

- **Resource exhaustion (noisy neighbour).** One tenant deploys workloads without resource limits and degrades the availability of the other tenant.
- **Unauthorised access to the control plane.** Traffic reaches the API server or the kubelet from outside the intended communication paths.

4.2 Core security controls

The security scope of this project consists of the following four mechanisms: OpenNebula security groups, Kubernetes RBAC, NetworkPolicy, and Pod Security Standards.

Threat	Control	Layer
Unauthorised control-plane access	OpenNebula security groups (default deny, explicit allow for API server, kubelet and CNI traffic)	IaaS
Cross-tenant data access	RBAC: per-tenant ServiceAccount, Role and RoleBinding scoped to the namespace; no ClusterRole granted to tenants	PaaS
Cross-tenant network access	NetworkPolicy: default-deny ingress and egress per namespace, with explicit intra-tenant exceptions	PaaS
Privilege escalation from a workload	Pod Security Standards enforced at the restricted level on tenant namespaces	PaaS

These four mechanisms are conceptually independent: RBAC governs what an identity may ask the API to do; NetworkPolicy governs which pods may talk to which; Pod Security Standards constrain what a pod may do at runtime; and OpenNebula security groups restrict traffic between the underlying virtual machines regardless of what happens inside Kubernetes. Together they already form a layered defence rather than a single point of enforcement.

4.3 Planned extensions (time permitting)

Beyond the core commitment above, we plan to add two further mechanisms if time allows. We list them separately because they address concerns not covered by the core controls, rather than duplicating them:

- **OPA/Gatekeeper admission policies.** Two custom constraints validated at admission time: every pod must declare CPU and memory requests/limits (mitigating noisy-neighbour resource exhaustion), and every pod must carry a tenant label consistent with its namespace (supporting auditability). This exercises admission control, a mechanism not otherwise present in the core design.
- **API server audit logging.** An audit policy recording requests against pod resources, giving the forensic visibility that a real multi-tenant platform would need for incident response.

5 Demonstration Workload

To make isolation observable, each tenant runs the same simple two-tier application: a small Python/Flask web front-end backed by a PostgreSQL database. The application deliberately carries no business complexity, it exists only to give the isolation mechanisms something concrete to protect and to make cross-tenant access attempts visible during the demo.

The database is backed by a PersistentVolumeClaim per tenant, which additionally exercises the block-storage abstractions covered in the course (volumes, PersistentVolume, PersistentVolumeClaim, StorageClass) and provides a natural target for demonstrating that Team Alpha cannot reach the storage or the database of Team Beta.

6 Expected Outcome and Deliverables

The final result is a reproducible platform, deployable from scratch on a single lab VM, together with evidence that each isolation boundary holds. The live demo will be structured as a sequence of attempted violations, each blocked by a different mechanism: a connection to the control plane on a non-permitted port refused by the OpenNebula security group; a `kubectl` request issued with the Team Alpha `kubeconfig` against Team Beta resources rejected by RBAC; direct pod-to-pod traffic between the two namespaces dropped by NetworkPolicy; and a privileged pod rejected by Pod Security Standards. If the planned extensions are completed, the sequence will also include a pod without resource limits rejected by Gatekeeper.

The deliverables will be: this proposal, a final report describing design and implementation, the complete source code and manifests in a public GitHub repository, and a README with step-by-step instructions to reproduce the deployment.

7 Relation to Course Topics

The project deliberately spans both the IaaS and PaaS sides of the course, and touches the following topics covered in lectures and labs: hardware virtualisation and hypervisors (KVM under OpenNebula); management of cloud resources through an IaaS platform; OS-level virtualisation and containers; L2/L3 networking in virtualised environments (OpenNebula virtual networks and the Kubernetes CNI); storage abstractions (volumes and persistent claims); orchestration with Kubernetes (pods, services, namespaces, volumes); and security applied to container and Kubernetes environments (RBAC, NetworkPolicy, Pod Security Standards, admission control).